

Esercizi d'esame per categoria — Seconda parte

Raccolta di **tutti i 71 esercizi** dei 18 esami presenti in [Esami/Seconda parte](#) (dal 14-06-16 al 19-06-06), raggruppati per **tecnica risolutiva** anziché per data. Ogni esercizio riporta la traccia riassunta, l'idea della soluzione ottima con la formula/ricorrenza chiave, la complessità e il riferimento all'esame di provenienza (es. [\[15-07-14 · Es.4\]](#)).

L'obiettivo è il ritrovamento rapido: se in un esame trovi un problema "di quel tipo", vai alla sotto-categoria corrispondente e confronti con i casi già visti.

Indice

A. Ricorrenze e analisi di complessità — 14 esercizi

- [A.1 Ricorrenze risolvibili con i teoremi \(master / partizione bilanciata\)](#)
- [A.2 Ricorrenze con chiamate di dimensioni diverse \(metodo di sostituzione\)](#)
- [A.3 Ricorrenze che richiedono il rafforzamento dell'ipotesi induttiva](#)
- [A.4 Analisi di complessità di codice / di algoritmi noti su input speciali](#)

B. Programmazione dinamica e memoization — 21 esercizi

- [B.1 DP su stringhe: LCS e parenti \(allineamento, supersequenza, shuffle, conteggio\)](#)
- [B.2 DP su stringhe: palindromi](#)
- [B.3 DP di partizionamento / taglio ottimo \(intervalli su stringa\)](#)
- [B.4 DP tipo zaino: subset-sum, resto, partizioni](#)
- [B.5 DP di selezione su sequenza con vincoli](#)
- [B.6 DP di conteggio \(numero di soluzioni\)](#)
- [B.7 Memoization su strutture non ordinabili \(quando la DP "in avanti" non basta\)](#)

C. Grafi — 13 esercizi

- [C.1 Visite DFS/BFS: connessione, cicli, bipartizione](#)
- [C.2 Raggiungibilità, vertici principali, SCC, ordinamento topologico](#)
- [C.3 Cammini minimi](#)
- [C.4 DP/memoization su DAG \(cammini massimi, etichette, probabilità\)](#)

D. Flusso massimo e matching — 11 esercizi

- [D.1 Assegnamento bipartito con capacità \(lo schema base\)](#)
- [D.2 Reti con gadget: vincoli di quota e di diversità](#)
- [D.3 Matching su un solo insieme \(accoppiamento di elementi fra loro\)](#)
- [D.4 Cammini arco-disgiunti](#)
- [D.5 Teoria del flusso e ricalcolo incrementale](#)

E. Greedy — 7 esercizi

- [E.1 Ordinamento per rapporto / per valore \(dimostrazione per scambio\)](#)
- [E.2 Intervalli: copertura minima](#)
- [E.3 Greedy che NON funziona \(controesempi da saper costruire\)](#)

F. Backtracking ed enumerazione — 6 esercizi

G. Array, vettori, ordinamento, divide et impera — 6 esercizi

Appendice: indice per esame

A. Ricorrenze e analisi di complessità

Schema di lavoro standard. (1) Indovina il limite guardando il termine non ricorsivo (dà sempre un Ω immediato) e la "somma delle frazioni" $\sum_i a_i/b_i^\beta$ delle chiamate. (2) Se $\sum_i a_i/b_i^\beta < 1$ il costo è dominato dal termine non ricorsivo; se = 1 spesso compare un fattore $\log n$. (3) Dimostra il limite superiore per sostituzione: caso base, ipotesi induttiva, passo induttivo, e ricava la condizione su c . (4) Attenzione ai **casi base**: vanno verificati per tutti gli n sotto la soglia della ricorrenza.

A.1 Ricorrenze risolubili con i teoremi

$$T(n) = 2T(n/8) + \sqrt[3]{n} - \text{limite superiore} \quad [14-06-16 \cdot \text{Es.1}]$$

Traccia. Trovare un limite superiore per $T(n) = 2T(n/8) + \sqrt[3]{n}$ e dimostrarlo formalmente.

Soluzione. Master theorem / teorema delle ricorrenze con partizione bilanciata: $a = 2, b = 8, \beta = 1/3$, e $\log_8 2 = 1/3 = \beta \rightarrow$ **caso 2**:

$$T(n) = \Theta(\sqrt[3]{n} \log n)$$

Per sostituzione con ipotesi $T(n) \leq c\sqrt[3]{n} \log n$, il passo induttivo chiude con $c \geq 1/3$. I casi base da verificare sono $n = 2, \dots, 15$.

$$T(n) = 2T(n/8) + \sqrt[3]{n} - \text{limite inferiore} \quad [15-07-14 \cdot \text{Es.1}]$$

Traccia. Stessa ricorrenza, ma stavolta va dimostrato il limite **inferiore**.

Soluzione. $T(n) = \Omega(\sqrt[3]{n} \log n)$. La struttura della dimostrazione è identica, ma con la disuguaglianza rovesciata ($T(n) \geq c\sqrt[3]{n} \log n$); la condizione finale diventa $1 - 3c \geq 0$, cioè $c \leq 1/3$. Insieme a [14-06-16 · Es.1] dà Θ .

$$T(n) = 2T(\lfloor n/\sqrt{2} \rfloor - 5) + n^{\pi/2} \quad [16-07-06 \cdot \text{Es.1}]$$

Traccia. Limite superiore per una ricorrenza con un -5 dentro l'argomento e un esponente irrazionale.

Soluzione. Il termine -5 si può eliminare per maggiorazione: $T(n) \leq 2T(n/\sqrt{2}) + n^{\pi/2}$. Teorema delle ricorrenze lineari con partizione bilanciata:

$$\alpha = \log_{\sqrt{2}} 2 = 2, \quad \beta = \frac{\pi}{2} \approx 1.57$$

Poiché $\alpha > \beta$, si ha $T(n) = O(n^2)$.

A.2 Ricorrenze con chiamate di dimensioni diverse

Tutte queste hanno la stessa struttura: k chiamate di dimensione diversa + un termine polinomiale n^β . Il trucco è sempre lo stesso: sommare le frazioni $\sum_i a_i/b_i^\beta$; se la somma è < 1 , il risultato è $\Theta(n^\beta)$ e il limite superiore si dimostra per sostituzione trovando il c giusto.

$$T(n) = T(n/\sqrt{2}) + T(n/\sqrt{4}) + T(n/\sqrt{8}) + T(n/\sqrt{16}) + n^2 \quad [15-02-02 \cdot \text{Es.1}]$$

Soluzione. $\Omega(n^2)$ è immediato dal termine non ricorsivo. Per $O(n^2)$, le frazioni sommano a

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = \frac{15}{16} < 1 \quad \Rightarrow \quad \frac{15}{16}cn^2 + n^2 \leq cn^2 \iff c \geq 16$$

Risultato: $T(n) = \Theta(n^2)$. (Casi base: $T(2), T(3)$.)

$$T(n) = T(n/\sqrt[3]{2}) + T(n/\sqrt[3]{5}) + T(n/\sqrt[3]{7}) + n^3 \quad [15-06-08 \cdot \text{Es.1}]$$

Soluzione. $\Omega(n^3)$ immediato. Le frazioni cubiche sommano a

$$\frac{1}{2} + \frac{1}{5} + \frac{1}{7} = \frac{59}{70} < 1 \quad \Rightarrow \quad c \geq \frac{70}{11}$$

Risultato: $T(n) = \Theta(n^3)$.

$$T(n) = 2T(n/2) + 4T(n/4) + 15T(n/8) + n^2, \text{ con } T(n) = 1 \text{ per } n \leq 8 \quad [17-06-29 \cdot \text{Es.1}]$$

Soluzione. $\Omega(n^2)$ immediato. Frazioni:

$$\frac{2}{4} + \frac{4}{16} + \frac{15}{64} = \frac{63}{64} < 1 \quad \Rightarrow \quad \frac{63}{64}cn^2 + n^2 \leq cn^2 \iff c \geq 64$$

Risultato: $T(n) = \Theta(n^2)$.

$$T(n) = 3T(n/3) + 6T(n/6) + 54T(n/12) + n^2, \text{ con } T(n) = 1 \text{ per } n \leq 12 \text{ [17-09-04 · Es.1]}$$

Soluzione. Identico al precedente. Frazioni:

$$\frac{3}{9} + \frac{6}{36} + \frac{54}{144} = \frac{7}{8} < 1 \quad \Rightarrow \quad c \geq 8$$

Risultato: $T(n) = \Theta(n^2)$.

$$T(n) = T(n/15) + T(n/10) + 2T(n/6) + \sqrt{n} \text{ [16-08-29 · Es.1]}$$

Soluzione. Per sostituzione con $T(n) \leq cn$, le frazioni sommano a

$$\frac{1}{15} + \frac{1}{10} + \frac{2}{6} = \frac{1}{2} < 1 \quad \Rightarrow \quad \frac{cn}{2} + \sqrt{n} \leq cn \iff c \geq 2$$

quindi $T(n) = O(n)$.

Limite più stretto: maggiorando tutte le chiamate con la più grande, $T(n) \leq 4T(n/6) + \sqrt{n}$, e col master theorem si ottiene

$$T(n) = O(n^{\log_6 4}) \approx O(n^{0.77})$$

⚠ Attenzione ai casi base: per $n < 15$ le chiamate generano $T(0)$.

A.3 Ricorrenze che richiedono il rafforzamento dell'ipotesi induttiva

Segnale d'allarme: il termine non ricorsivo è una **costante** (o comunque di ordine inferiore al limite che vuoi dimostrare) e le frazioni sommano **esattamente a 1**. Con l'ipotesi $T(n) \leq cn$ il passo induttivo produce $cn + 1 \not\leq cn$ e non chiude.

Rimedio: rafforzare l'ipotesi a $T(n) \leq cn - b$, con $b > 0$; il $-b$ assorbe il termine costante.

$$T(n) = \min_{1 \leq k \leq n-1} \{T(k) + T(n-k)\} + 1 \text{ [14-07-21 · Es.1]}$$

Traccia. Trovare un limite superiore alla ricorrenza (il $\min$ su tutte le partizioni possibili).

Soluzione. $T(n) = O(n)$ — in realtà $T(n)$ cresce come $2n - 1$.

Con l'ipotesi $T(n) \leq cn$ la dimostrazione fallisce per via del $+1$. Con l'ipotesi rafforzata $T(n) \leq cn - b$:

$$T(n) \leq (ck - b) + (c(n - k) - b) + 1 = cn - 2b + 1 \leq cn - b \iff b \geq 1$$

e i casi base danno $c \geq 2$.

Coppia di ricorrenze (limiti sup. e inf.) [16-01-26 · Es.1]

$$(a) T(n) = \frac{1}{2}(T(n-1) + T(3n/4)) + n \Rightarrow \Theta(n).$$

$\Omega(n)$ è immediato; per $O(n)$ l'ipotesi $T(n) \leq cn$ funziona senza rafforzamenti:

$$\frac{1}{2}(cn - c) + \frac{1}{2} \cdot \frac{3cn}{4} + n = \frac{7}{8}cn + n \leq cn \iff c \geq 8$$

$$(b) T(n) = T(n/2) + T(n/4) + T(n/6) + T(n/12) + 1 \Rightarrow \Theta(n).$$

Qui le frazioni sommano **esattamente a 1**:

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{6} + \frac{1}{12} = 1$$

con $T(n) \leq cn$ si ottiene $cn + 1 \not\leq cn$. Serve l'ipotesi rafforzata $T(n) \leq cn - b$, che chiude con $b \geq 1/3$.

A.4 Analisi di complessità di codice o di algoritmi noti

Analisi della funzione `crazy(n)` [17-01-24 · Es.1]

Traccia. Dato il codice di `crazy(n)` (due cicli annidati `for i ← 1 to n, for j ← i to n` seguiti da `crazy(n/4) + crazy(n/2) + crazy(n/4)`), trovare limite superiore e inferiore.

Soluzione. Prima si **estrae la ricorrenza dal codice**: i due cicli annidati costano $\Theta(n^2)$, quindi

$$T(n) = \begin{cases} 2T(\lfloor n/4 \rfloor) + T(\lfloor n/2 \rfloor) + n^2 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

$\Omega(n^2)$ è immediato; per $O(n^2)$:

$$\frac{2}{16} + \frac{1}{4} = \frac{3}{8} \implies \frac{3}{8}cn^2 + n^2 \leq cn^2 \iff c \geq \frac{8}{5}$$

Risultato: $T(n) = \Theta(n^2)$.

Comportamento di MergeSort / QuickSort / InsertionSort su un vettore "5-crescente" [16-01-07 · Es.1]

Traccia. Il vettore V rispetta $V[j] > V[i]$ per ogni $j \geq i + 5$ (cioè è "quasi ordinato": ogni elemento può essere fuori posto al massimo di 4 posizioni). Analizzare il costo dei tre algoritmi su questo input.

Soluzione.

- **MergeSort**: $\Theta(n \log n)$ sempre — la struttura dell'input non lo influenza.
- **QuickSort non probabilistico** (pivot = primo elemento): il pivot è più piccolo di tutti gli elementi tranne al più 4 → la partizione è massimamente sbilanciata → $T(n) = T(n-5) + O(n) \Rightarrow \Theta(n^2)$ in ogni caso (non solo nel caso pessimo).
- **InsertionSort**: il ciclo interno esegue al massimo 5 spostamenti → costo del ciclo interno costante → $\Theta(n)$. È il migliore dei tre su questo input.

B. Programmazione dinamica e memoization

Categoria più frequente in assoluto (~30% degli esercizi), quasi sempre nelle posizioni 3 o 4 (le più pesanti in punti). Da scrivere sempre: (1) definizione **a parole** del significato della tabella, (2) **formula ricorsiva** coi casi base, (3) codice (iterativo o con memoization), (4) complessità, (5) eventualmente la funzione di ricostruzione della soluzione.

B.1 DP su stringhe: LCS e parenti

Sottosequenza crescente e a caratteri distinti [14-12-19 · Es.3]

Traccia. Data una stringa su alfabeto A-Z, trovare la più lunga sottosequenza con caratteri **crescenti e tutti distinti**.

Soluzione furba. È esattamente $\text{LCS}(S, \text{"ABC...Z"})$: la LCS con l'alfabeto ordinato è per costruzione crescente e senza ripetizioni. Costo $\Theta(26 \cdot n) = \Theta(n)$.

Formula esplicita. $M[i, c]$ = lunghezza massima usando i primi i caratteri e solo caratteri $\leq c$:

$$M[i, c] = \begin{cases} 0 & i = 0 \\ M[i-1, c] & S[i] > c \\ \max \{ M[i-1, c], M[i-1, S[i]-1] + 1 \} & S[i] \leq c \end{cases}$$

Allineamento di due stringhe con soli trattini [14-12-19 · Es.4]

Traccia. Date $P[1..n]$ e $T[1..m]$, calcolare il **minimo numero di trattini** (–) da inserire per allinearle (versione semplificata dell'edit distance: sono ammesse solo inserzioni).

$$D[i, j] = \begin{cases} j & i = 0 \\ i & j = 0 \\ D[i-1, j-1] & P[i] = T[j] \\ \min \{ D[i-1, j], D[i, j-1] \} + 1 & \text{altrimenti} \end{cases}$$

Complessità. $O(nm)$.

Supersequenza comune minimale (SCS) [17-01-10 · Es.2]

Traccia. P è supersequenza di T se T è sottosequenza di P . Trovare la lunghezza della **più piccola supersequenza comune** di P e T . (Es.: $SCS(AB, BC) = ABC$, di lunghezza 3.)

$$D[i, j] = \begin{cases} i & j = 0 \\ j & i = 0 \\ D[i-1, j-1] + 1 & P[i] = T[j] \\ \min \{ D[i-1, j], D[i, j-1] \} + 1 & \text{altrimenti} \end{cases}$$

Complessità. $\Theta(nm)$. È il **duale della LCS**: vale sempre

$$|SCS(P, T)| = n + m - |LCS(P, T)|$$

Shuffle / mescolamento di due stringhe [16-01-26 · Es.3]

Traccia. Date X, Y, Z , dire se Z è un **mescolamento** di X e Y (cioè ottenibile intercalando i caratteri di X e Y mantenendone l'ordine relativo).

$B[i, j] = \text{true}$ sse il prefisso $Z[1..i+j]$ è mescolanza di $X[1..i]$ e $Y[1..j]$:

$$\begin{aligned} B[0, 0] &= \text{true} \\ B[i, j] &= (X[i] = Z[i+j] \wedge B[i-1, j]) \vee (Y[j] = Z[i+j] \wedge B[i, j-1]) \end{aligned}$$

Complessità. $O(nm)$.

Contare le occorrenze di P come sottosequenza di T [16-08-29 · Es.4]

Traccia. Quante volte il pattern $P[1..m]$ compare come **sottosequenza** (non contigua) del testo $T[1..n]$?

$$C[p, t] = \begin{cases} 0 & p > t \quad (\text{pattern più lungo del testo residuo}) \\ 1 & p = 0 \quad (\text{pattern vuoto: un solo modo}) \\ C[p, t-1] & P[p] \neq T[t] \\ C[p-1, t-1] + C[p, t-1] & P[p] = T[t] \quad (\text{uso } T[t] \text{ oppure lo ignoro}) \end{cases}$$

Risposta. $C[m, n]$. **Complessità.** $O(mn)$.

B.2 DP su stringhe: palindromi

⚠ **Distinzione da tenere a mente:** la **sottosequenza** palindroma (caratteri non contigui) e la **sottostringa** palindroma (caratteri contigui) hanno formule diverse. Confinderle è l'errore classico.

Sottosequenza palindroma massimale (LPS) [14-07-21 · Es.4]

Traccia. Data una stringa, trovare la lunghezza della più lunga **sottosequenza** palindroma. Estensione: stamparla.

$$L[i, j] = \begin{cases} 0 & j < i \\ 1 & i = j \\ L[i+1, j-1] + 2 & s[i] = s[j] \\ \max \{ L[i+1, j], L[i, j-1] \} & \text{altrimenti} \end{cases}$$

Complessità. $O(n^2)$. **Alternativa:** $LPS(s) = LCS(s, reverse(s))$. La stampa si fa con una `printRec` che ripercorre la matrice.

Sottostringa palindroma massimale (contigua) [15-09-01 · Es.4]

Traccia. Come sopra, ma la sottostringa deve essere **contigua**.

$$M[i, j] = \begin{cases} j - i + 1 & j - i + 1 \leq 1 \\ j - i + 1 & s[i] = s[j] \wedge M[i + 1, j - 1] = j - i - 1 \quad (\text{l'interno è tutto palindromo}) \\ \max \{ M[i, j - 1], M[i + 1, j] \} & \text{altrimenti} \end{cases}$$

Complessità. $O(n^2)$.

Minima decomposizione di una stringa in palindromi [17-01-24 · Es.4]

Traccia. Ogni stringa può essere spezzata in una sequenza di sottostringhe palindrome. Trovare il **minimo numero** di pezzi. (Es.: "BUBBASEESABANANA" $\rightarrow$ 3, "AFISICACISIFA" $\rightarrow$ 1.)

Soluzione $O(n^3)$ — DP di partizionamento su matrice:

$$D[i, j] = \begin{cases} 1 & i \geq j \\ 1 & S[i] = S[j] \wedge D[i + 1, j - 1] = 1 \\ \min_{i \leq k < j} \{ D[i, k] + D[k + 1, j] \} & \text{altrimenti} \end{cases}$$

Soluzione $O(n^2)$ — questa è quella da preferire. Si pre-calcola in $\Theta(n^2)$ la matrice booleana $P[i, j] = \text{true}$ sse $S[i..j]$ è palindroma:

$$P[i, j] = \begin{cases} \text{true} & i \geq j \\ P[i + 1, j - 1] \wedge S[i] = S[j] & i < j \end{cases}$$

poi si usa un **vettore** invece di una matrice, dove $D[j]$ è il minimo numero di palindromi che compongono il prefisso $S[1..j]$:

$$D[j] = \begin{cases} 0 & j = 0 \\ \min_{1 \leq i \leq j \wedge P[i, j]} \{ D[i - 1] + 1 \} & j > 0 \end{cases}$$

Risposta. $D[n]$.

B.3 DP di partizionamento / taglio ottimo

Schema $\min_{i < k < j} \{ D[i, k] + D[k, j] \} + \text{costo}(i, j)$ — tipico dei problemi "scegli dove spezzare" (matrix-chain, taglio di una stringa/asta). Complessità tipica $O(n^3)$.

Taglio ottimo di una stringa [16-01-07 · Es.4]

Traccia. Una stringa di lunghezza L va tagliata nelle posizioni $V[1..n]$. Il costo di un taglio è pari alla **lunghezza del pezzo che si sta tagliando** (quindi l'ordine dei tagli conta). Minimizzare il costo totale.

Soluzione. Si aggiungono le sentinelle $V[0] = 0$ e $V[n + 1] = L$.

$$D[i, j] = \begin{cases} 0 & j = i + 1 \quad (\text{nessun taglio in mezzo}) \\ \min_{i < k < j} \{ D[i, k] + D[k, j] \} + (V[j] - V[i]) & \text{altrimenti} \end{cases}$$

Risposta. $D[0, n + 1]$. **Complessità.** $O(n^3)$.

(Vedi anche la decomposizione in palindromi [17-01-24 · Es.4], che usa lo stesso schema.)

B.4 DP tipo zaino: subset-sum, resto, partizioni

Salvadanaio: knapsack con ripetizione, in **minimizzazione** [15-02-02 · Es.4]

Traccia. Un salvadanaio pesa complessivamente T . Ci sono n tipi di moneta, ognuno con peso $p[i]$ e valore $v[i]$, disponibili in quantità illimitata. Trovare il **minimo valore** compatibile con il peso T .

$$M(t, i) = \begin{cases} 0 & t = 0 \\ +\infty & t < 0 \vee (i = 0 \wedge t > 0) \\ \min \{ M(t - p[i], i) + v[i], M(t, i - 1) \} & \text{altrimenti} \end{cases}$$

Nota il termine $M(t - p[i], i)$, con la i che **non decresce**: è ciò che modella l'uso ripetuto della stessa moneta. **Complessità.** $O(nT)$ (pseudopolinomiale).

Resto con al più k monete [15-06-08 · Es.3]

Traccia. Dati n tagli $v[1..n]$ (disponibilità illimitata), dire se è possibile dare un resto R usando **al più k monete**.

$$P[i, r, j] = \begin{cases} \text{false} & r < 0 \\ \text{true} & r = 0 \\ \text{false} & r > 0 \wedge (i = 0 \vee j = 0) \\ P(i, r - v[i], j - 1) \vee P(i - 1, r, j) & \text{altrimenti} \end{cases}$$

Complessità. $O(nRk)$.

Calchetto: partizione in due squadre bilanciate [15-07-14 · Es.4]

Traccia. $2n$ giocatori con "bravura" $b[i]$. Dividerli in **due squadre da n giocatori** ciascuna con la **stessa bravura totale**.

Soluzione. È un subset-sum con **doppio vincolo** (somma e cardinalità). $C(m, s, k) =$ "posso ottenere somma m usando esattamente s giocatori scelti fra i primi k ?"

$$C(m, s, k) = C(m - b[k], s - 1, k - 1) \vee C(m, s, k - 1)$$

casi base: $C(0, 0, k) = \text{true}$; **false** se $m < 0 \vee s < 0$; **false** se $k = 0 \wedge (m > 0 \vee s > 0)$

Problema da risolvere. $C(\frac{M}{2}, n, 2n)$, dove $M = \sum_i b[i]$. **Complessità.** $O(M \cdot n^2)$.

Somma massimale (k, w) -vincolata [18-06-04 · Es.4]

Traccia. Dato X di n interi positivi distinti e due interi k, w : trovare il massimo valore

$$v = \max_{S \subseteq \{1, \dots, n\} \wedge |S| \leq k} \sum_{i \in S} X[i] \quad \text{tale che} \quad v \leq w$$

(Es.: $X = [12, 13, 11, 6, 23, 19]$, $w = 27$, $k = 3 \rightarrow$ risposta $25 = 12 + 13 = 19 + 6$.)

$$DP[i][s][v] = \begin{cases} 0 & v = 0 \vee i = 0 \vee s = 0 \\ DP[i - 1][s][v] & X[i] > v \\ \max \{ DP[i - 1][s][v], DP[i - 1][s - 1][v - X[i]] + X[i] \} & \text{altrimenti} \end{cases}$$

Complessità. $O(nkw) = O(n^2w)$ (pseudopolinomiale, essendo $k \leq n$).

Somma di quadrati (parte DP) [16-01-26 · Es.4a]

Traccia. Minimo numero di quadrati perfetti la cui somma è n .

⚠ Il greedy "prendi sempre il quadrato più grande" **non funziona**: per 18 darebbe $16 + 1 + 1$ (3 termini), mentre l'ottimo è $9 + 9$ (2 termini).

$$Q(n) = \begin{cases} 0 & n = 0 \\ \min_{1 \leq t \leq \sqrt{n}} \{ Q(n - t^2) + 1 \} & \text{altrimenti} \end{cases}$$

Complessità. $\Theta(n\sqrt{n})$. (La parte (b), la stampa di tutte le decomposizioni, è in F .)

B.5 DP di selezione su sequenza con vincoli

Schema "prendo / non prendo l'elemento i ", con uno **stato aggiuntivo** che ricorda il vincolo (distanza minima, quanti consecutivi ho già preso, carburante residuo...).

Auto elettrica: pianificazione dei rifornimenti [14-06-16 · Es.4]

Traccia. Un'auto con autonomia A percorre una strada con n distributori; il distributore i è a distanza $d[i]$ e ha un tempo di rifornimento unitario $t[i]$ (fare g unità di carburante costa $g \cdot t[i]$). Minimizzare il **tempo totale speso nei rifornimenti** arrivando in fondo.

$$T[i, a] = \begin{cases} 0 & i = n \\ \min_{\substack{0 \leq g \leq A-a \\ a+g \geq d[i+1]}} \{ T[i+1, a+g] + g \cdot t[i] \} & \text{altrimenti} \end{cases}$$

dove a è il carburante residuo all'arrivo al distributore i ; il vincolo $a + g \geq d[i+1]$ garantisce di poter raggiungere il distributore successivo. **Complessità.** $O(n \cdot A^2)$.

k -sottosequenza massimale: al più k elementi **consecutivi** [15-12-17 · Es.2]

Traccia. Dato un vettore V , massimizzare la somma degli elementi scelti, non potendo però prenderne più di k **consecutivi**.

Soluzione. $D[i, j]$ = valore massimo usando i primi i elementi, avendo ancora j "prese consecutive" disponibili.

$$D[i, j] = \begin{cases} 0 & i = 0 \\ D[i-1, k] & j = 0 \quad (\text{costretto a saltare; il contatore si ricarica}) \\ \max \{ D[i-1, k], D[i-1, j-1] + V[i] \} & j > 0 \quad (\text{salto} \rightarrow \text{ricarico } k; \text{ prendo} \rightarrow \text{scalo } j) \end{cases}$$

Risposta. $D[n, k]$. **Complessità.** $O(nk)$.

Route 66 / "The Donald": comizi a distanza minima D [16-07-06 · Es.3]

Traccia. n città lungo una strada: la città i è al miglio $m[i]$ e vale $e[i]$ elettori. Due comizi devono distare **almeno** D miglia. Massimizzare gli elettori raggiunti.

$$best[j] = \begin{cases} e[1] & j = 1 \\ \max \left\{ best[j-1], e[j] + \max_{\substack{1 \leq i < j \\ m[j]-m[i] \geq D}} best[i] \right\} & j > 1 \end{cases}$$

Risposta. $best[n]$. **Complessità.** $O(n^2)$ (riducibile con ricerca binaria sull'indice ammissibile).

B.6 DP di conteggio

Domino $2 \times n$: contare le disposizioni [15-12-17 · Es.3a]

Traccia. In quanti modi si può ricoprire una scacchiera $2 \times n$ con tessere di domino 2×1 ?

Soluzione. L'ultima colonna è coperta o da una tessera verticale (resta $2 \times (n-1)$) o da due tessere orizzontali (resta $2 \times (n-2)$):

$$D(n) = D(n-1) + D(n-2) \quad \implies \quad D(n) = \text{Fib}(n+1)$$

Con DP/memoization: $O(n)$. (La parte (b), stampare tutte le disposizioni, è in F.)

Cammini su griglia con budget [19-06-06 · Es.B3]

Traccia. Scacchiera $n \times n$, pedone da $(1, 1)$ a (n, n) , si muove solo in **basso o a destra**. Ogni cella ha un costo $P[r][c] \in [0, 10]$. Contare i cammini **distinti** di costo $\leq B$.

$$DP[r][c][b] = \begin{cases} 0 & b < 0 \vee r = 0 \vee c = 0 \\ 1 & r = 1 \wedge c = 1 \wedge b \geq 0 \\ DP[r-1][c][b-P[r][c]] + DP[r][c-1][b-P[r][c]] & \text{altrimenti} \end{cases}$$

Trucco chiave (fa la differenza tra polinomiale e pseudopolinomiale). La terza dimensione non va allocata su B , ma su

$$size = \min(B, 10 \cdot (2n - 2))$$

perché ogni cammino attraversa esattamente $2n - 1$ celle, quindi un budget superiore a $10(2n - 2)$ è indistinguibile da quel valore. Così $size = O(n)$ e la complessità è $O(n^3)$, **polinomiale** (non pseudopolinomiale). Spazio riducibile a $O(n^2)$ non dovendo ricostruire la soluzione.

B.7 Memoization su strutture non ordinabili

Quando le dipendenze fra sottoproblemi **non hanno un ordine topologico ovvio** (si va avanti e indietro), la DP iterativa non si può scrivere: si usa la **memoization** ricorsiva.

Scale e serpenti (Snakes & Ladders) [17-06-29 · Es.2]

Traccia. Scacchiera $n \times n$ numerata a serpentina, dado a 6 facce. $V[i] > i$ = scala, $V[i] < i$ = serpente, $V[i] = i$ = casella normale. Minimo numero di **tiri di dado** per arrivare alla casella n^2 .

Soluzione A (la migliore da scrivere): BFS. Si costruisce un grafo in cui ogni casella ha un arco verso ciascuna delle 6 caselle successive, reindirizzate alla destinazione se sono scale/serpenti. Il minimo numero di tiri è la **distanza BFS** da 1 a n^2 (archi non pesati!). Costo $O(|V| + |E|) = O(n^2 + 6n^2) = O(n^2)$.

⚠ Usare Dijkstra qui costa $O(|V|^2) = O(n^4)$: è **sbagliato come scelta**, non come risultato.

Soluzione B: memoization.

$$DP[i] = \begin{cases} 0 & i \geq n^2 \\ DP[V[i]] & i < n^2 \wedge V[i] \neq i \quad (\text{scala/serpente: spostamento automatico, non costa un tiro}) \\ \min_{1 \leq k \leq 6} \{ DP[i+k] \} + 1 & i < n^2 \wedge V[i] = i \end{cases}$$

Non è DP "pura" perché scale e serpenti portano avanti e **indietro** nel tabellone.

⚠ Le soluzioni greedy ("segui sempre le scale, evita i serpenti") **non funzionano**: possono esserci 6 caselle consecutive tutte con serpenti, e possono esistere cicli scala-serpente.

C. Grafi

C.1 Visite DFS/BFS: connessione, cicli, bipartizione

Contare le componenti connesse che sono alberi [14-06-16 · Es.2]

Traccia. Dato un grafo non orientato, contare quante componenti connesse sono **anche alberi** (cioè acicliche).

Soluzione. Una DFS per componente; durante la visita si controlla l'aciclicità: si trova un ciclo se si incontra un nodo già visitato **diverso dal padre** p (l'arco verso il padre non conta). Si incrementa il contatore solo se la componente non ha cicli. **Complessità.** $O(m + n)$.

Wrestler buoni e cattivi (bipartizione / 2-colorabilità) [17-01-24 · Es.2]

Traccia. Grafo delle rivalità fra wrestler. Si possono dividere in due gruppi non vuoti ("buoni" e "cattivi") in modo che **non ci siano rivalità interne ai gruppi**?

Soluzione. È esattamente il problema della **bicolorazione**: possibile $\iff$ il grafo è **bipartito**. DFS che assegna il colore c al nodo corrente e ricorre sui vicini con $1 - c$; se un vicino è già colorato **con lo stesso colore** $\rightarrow$ **false**. Va lanciata da ogni nodo non ancora visitato (il grafo può essere disconnesso). **Complessità.** $O(m + n)$.

Ordine di eliminazione che mantiene la connessione [15-06-08 · Es.2]

Traccia. Dato un grafo non orientato **connesso**, trovare un ordine $v_1, \dots, v_n$ tale che eliminando i nodi in quell'ordine il grafo resti sempre connesso.

Soluzione. Una DFS che restituisce i nodi in ordine di **tempo di fine crescente**. Implementativamente è come l'ordinamento topologico, ma i nodi si accodano in una **coda** invece che in una pila. Il nodo con tempo di fine più basso è una foglia dell'albero DFS (o comunque non è un punto di articolazione rispetto ai rimanenti), quindi rimuoverlo non disconnette. **Complessità.** $O(m + n)$.

C.2 Raggiungibilità, vertici principali, SCC, ordinamento topologico

Esiste un vertice principale? [14-07-21 · Es.2]

Traccia. Un vertice v è **principale** se da esso si raggiungono tutti gli altri. (a) Scrivere `isPrincipal(G, v)`. (b) Dire se ne esiste almeno uno.

Soluzione.

(a) Una DFS da v : il vertice è principale sse la visita raggiunge tutti i nodi. $O(m + n)$.

(b) **Il candidato è uno solo:** si esegue una DFS sul grafo **trasposto** G^T calcolando i tempi di fine e si prende l'**ultimo nodo chiuso** v (equivalentemente: il primo nodo dell'ordinamento topologico di G). Poi si verifica con `isPrincipal(G, v)`. $O(m + n)$.

Correttezza (per assurdo): se esiste un vertice principale, deve essere quello — nessun altro nodo può "vederlo" a sua volta senza contraddire l'ordine dei tempi di fine.

Contare i vertici principali [15-02-02 · Es.2]

Traccia. Quanti sono i vertici che raggiungono tutti gli altri?

Soluzione banale. DFS da ogni nodo: $O(mn)$.

Soluzione efficiente. I vertici principali, se esistono, formano **una sola componente fortemente connessa** (se u e v sono entrambi principali, si raggiungono a vicenda). Si calcolano le SCC con **Kosaraju**; solo la componente "sorgente" (la prima scoperta nell'ordine dei tempi di fine decrescenti) può contenerli. Si testa `isPrincipal` su **un solo nodo** di quella componente: se **true**, la risposta è la cardinalità della componente, altrimenti 0. **Complessità.** $O(m + n)$.

C.3 Cammini minimi

Alice, Bob e Carl: punto d'incontro evitando un nodo [15-07-14 · Es.2]

Traccia. Grafo pesato. Alice e Bob devono incontrarsi in un nodo u minimizzando la **somma** dei loro cammini, senza mai passare per il nodo di Carl.

Soluzione. Si "spegne" Carl ponendo $w[x, \text{Carl}] = +\infty$ per tutti gli **archi entranti** in Carl (equivalente a rimuoverlo dal grafo). Poi **due esecuzioni** di un algoritmo di cammini minimi (Dijkstra se i pesi sono non negativi), una da Alice e una da Bob, ottenendo $d_a[\cdot]$ e $d_b[\cdot]$. La risposta è

$$\min_{u \in V} \{ d_a[u] + d_b[u] \}$$

Complessità. $O(m \log n)$.

Cammino minimo in un albero binario "alloopato" [16-01-07 · Es.2]

Traccia. Un albero binario a cui sono stati aggiunti archi **da ogni foglia alla radice**. I pesi possono essere negativi, ma non ci sono cicli negativi. Trovare il cammino minimo da u a v .

Soluzione. Non serve Bellman-Ford: si sfrutta la struttura. Sia r la radice, e sia `dfsSearch(x, t, r)` una funzione che cerca t a partire da x **senza passare per r** .

- se v sta nel sottoalbero di u , il cammino è quello diretto nell'albero;

- altrimenti il cammino minimo passa **necessariamente per la radice**: $\text{dist}(u \rightarrow r) + \text{dist}(r \rightarrow v)$.

Poiché l'albero ha $m < 2n$ archi, il costo è $O(n)$.

C.4 DP/memoization su DAG

Il DAG (esplicito o implicito, come nel caso di un grafo con altitudini strettamente decrescenti) è il terreno naturale per la memoization su grafo: $D[u] = f(D[v] : v \in \text{adj}(u))$, calcolato con **una sola DFS complessiva**.

Snowboard: cammino discendente più lungo [17-09-04 · Es.3]

Traccia. Grafo con altitudini $z[u]$ e distanze $d[u, v]$. Trovare il cammino **più lungo** $P = u_1, \dots, u_k$ (lunghezza = $\sum_{i=2}^k d[u_{i-1}, u_i]$) lungo il quale l'altitudine è **strettamente decrescente**, cioè $z[u_{i-1}] > z[u_i]$ per ogni i .

Soluzione. Il vincolo $z[v] < z[u]$ rende il grafo implicitamente un **DAG**, quindi i sottoproblemi sono ben fondati.

$$D[u] = \begin{cases} 0 & \nexists v : (u, v) \in E \wedge z[v] < z[u] \\ \max_{\substack{v : (u, v) \in E \\ z[v] < z[u]}} \{d[u, v] + D[v]\} & \text{altrimenti} \end{cases}$$

Una sola DFS con memoization, lanciata da ogni nodo non ancora calcolato. Risposta: $\max_u D[u]$. **Complessità.** $O(m + n)$, contro l' $O(mn)$ della soluzione banale che rilancia una visita da ogni nodo.

Caccia al tesoro: numero atteso di premi [16-07-06 · Es.4]

Traccia. n premi; il premio i si trova autonomamente con probabilità $p[i]$. Un DAG di indizi: trovare i porta **con certezza** a tutti i premi raggiungibili da i . Calcolare il **numero atteso** di premi trovati.

Soluzione. Il trucco è ragionare sul complemento — la probabilità $x[u]$ che u **non** venga trovato:

$$x[u] = (1 - p[u]) \cdot \prod_{v \in \text{Pred}(u)} (1 - p[v])$$

cioè u non viene trovato se non lo si trova direttamente e non si trova nessuno dei suoi predecessori (che lo rivelerebbero).

Implementazione: si inizializza $x = \{1, \dots, 1\}$ e per ogni nodo u si lancia una DFS che moltiplica $x[v]$ di **ogni nodo** v **raggiungibile da** u (u compreso) per $(1 - p[u])$.

Risposta (per linearità del valore atteso):

$$\mathbb{E}[\#\text{premi}] = \sum_{u \in V} (1 - x[u])$$

Complessità. $O(nm)$.

Cammino con etichetta uguale a una stringa data [16-08-29 · Es.2]

Traccia. DAG con un'etichetta $\ell(v)$ su ogni nodo. Esiste un cammino la cui concatenazione di etichette è esattamente $A[1..n]$?

Soluzione (quella ufficiale). Da ogni nodo si lancia `checkLabels(G, i, A, n, i, u)`: se $\ell[u] = A[i]$ e $i = n \rightarrow \text{true}$; altrimenti si ricorre sui successori con $i + 1$. Complessità **superpolinomiale** — la soluzione ufficiale segnala esplicitamente che si può fare di meglio.

Come farla efficiente (utile saperlo): memoizzare sulla coppia (u, i) — "esiste un cammino che parte da u e produce il suffisso $A[i..n]$?" — riduce il costo a $O(n \cdot m)$.

D. Flusso massimo e matching

Il template da riprodurre a memoria. (1) Definisci i **nodi**: supersorgente s , superpozzo p , un livello di nodi per ogni "tipo" di entità. (2) Definisci **archi e capacità** — ogni vincolo del problema diventa una capacità. (3) Dichiara che una soluzione

valida esiste $\iff$ il flusso massimo raggiunge il valore target $|f^*|$. (4) Calcola $|V|$, $|E|$, $|f^*|$ e applica il limite di Ford-Fulkerson: $O(|f^*| \cdot (|V| + |E|))$. (5) Di' **dove si legge la soluzione** (di solito: gli archi del livello intermedio con flusso 1).

D.1 Assegnamento bipartito con capacità

Sensori wireless con k backup [14-07-21 · Es.3]

Traccia. n sensori nel piano; due sensori comunicano se distano meno di d . Ogni sensore vuole k backup fra quelli con cui comunica; ogni sensore può fare da backup per **al più** b altri sensori.

Rete. $s \rightarrow \text{sensore}_i$ (capacità k) $\rightarrow$ copia $_j$ per ogni j con cui i comunica (capacità 1) $\rightarrow p$ (capacità b).

Risposta. Valido $\iff$ il flusso massimo è $|f^*| = kn$. **Complessità.** $O(kn^3)$ con Ford-Fulkerson.

Piano di studi / regolamento [14-12-19 · Es.2]

Traccia. n corsi, k requisiti; il requisito i chiede m_i corsi presi dall'insieme S_i . Un corso **conta per al più un requisito**. In totale servono $t = \sum_i m_i$ corsi.

Rete. $s \rightarrow \text{requisito}_i$ (capacità m_i) $\rightarrow$ corso $_j$ per ogni $j \in S_i$ (capacità 1) $\rightarrow p$ (capacità 1 $\leftarrow$ è questa a garantire che un corso non copra due requisiti).

Risposta. Soddisfacibile $\iff$ il flusso massimo è t . **Complessità.** $O(t(n^2k + k^2n))$.

Parole formabili con cubi/dadi [15-02-02 · Es.3]

Traccia. Una parola di t lettere; n cubi, ognuno con 6 facce marcate con lettere. Ogni cubo può fornire **una sola** lettera. Si può formare la parola?

Rete. $s \rightarrow \text{posizione}_i$ della parola (capacità 1) $\rightarrow$ cubo $_j$ che contiene quella lettera (capacità 1) $\rightarrow p$ (capacità 1).

Risposta. Realizzabile $\iff$ il flusso è t . **Complessità.** $O(n^2)$.

Conferenza SIESTA: workshop [15-09-01 · Es.2]

Traccia. n società, la società i ha d_i dipendenti; m workshop, il workshop j ha capacità c_j ; la società i vuole i workshop in w_i ; **al più un dipendente per società per workshop**. Massimizzare le richieste soddisfatte.

Rete. $s \rightarrow \text{società}_i$ (capacità d_i) $\rightarrow$ workshop $_j \in w_i$ (capacità 1 $\leftarrow$ un solo dipendente per società per workshop) $\rightarrow p$ (capacità c_j).

Risposta. Il flusso massimo è il numero di richieste soddisfatte; è limitato da $\min(D, W, C)$.

Segherie: taglio di alberi su n foreste $\times m$ anni [17-01-24 · Es.3]

Traccia. w_{ij} alberi maturano nell'anno i nella foresta j ; un albero maturo può essere tagliato **in quell'anno o in uno qualunque successivo**; nell'anno i si possono tagliare al più u_i alberi in totale e al più v_{ij} nella foresta j . Massimizzare gli alberi tagliati.

Rete. Un nodo (a_i, f_j) per ogni coppia (anno, foresta) e un nodo a_i per ogni anno:

- $s \rightarrow (a_i, f_j)$ con capacità w_{ij} — gli alberi che maturano;
- $(a_i, f_j) \rightarrow (a_{i+1}, f_j)$ con capacità $+\infty \leftarrow$ *il gadget interessante*: modella il fatto che un albero maturo **resta tagliabile negli anni successivi**;
- $(a_i, f_j) \rightarrow a_i$ con capacità v_{ij} — limite dell'ente parco;
- $a_i \rightarrow p$ con capacità u_i — limite di personale.

Soluzione. Il flusso massimo è il numero di alberi tagliati; **quanti alberi tagliare per anno/foresta si legge sugli archi** $(a_i, f_j) \rightarrow a_i$. Con $|V| = nm + m + 2$, $|E| = 2nm + m$ e $U = \sum_i u_i$, il costo è $O(nmU)$.

D.2 Reti con gadget: vincoli di quota e di diversità

Quando il vincolo non è "quanti", ma "**di che tipo**" (esattamente $n/3$ per categoria; tutti di ditte diverse), serve un **livello intermedio di nodi-gadget**: la capacità sugli archi del gadget impone il vincolo.

Consiglio dei Corsi di Laurea: quote per categoria [16-08-29 · Es.3]

Traccia. n CdL, k docenti divisi in tre categorie (PO, PA, RU); ogni docente insegna in al più 3 CdL. Serve **un rappresentante per CdL**, con PO, PA e RU **in numero esattamente uguale** (n divisibile per 3).

Rete. $s \rightarrow \{PO, PA, RU\}$ (3 nodi-categoria, capacità $n/3$ ciascuno $\leftarrow$ **è questo livello a imporre le quote**) $\rightarrow$ docente della categoria (capacità 1) $\rightarrow$ CdL in cui insegna (capacità 1) $\rightarrow p$ (capacità 1).

Risposta. Fattibile $\iff$ il flusso è n . Con $|V| = n + k + 5$ e $|E| \leq n + 4k + 3$, il costo è $O(nk + k^2)$.

CERN: strumenti di ditte diverse [17-06-29 · Es.4]

Traccia. n proprietà fisiche $P = \{p_1, \dots, p_n\}$, m strumenti $S = \{s_1, \dots, s_m\}$ (lo strumento i misura un sottoinsieme di proprietà), t ditte $D = \{d_1, \dots, d_t\}$. Ogni proprietà va misurata da **almeno r strumenti**, e tutti gli r strumenti devono essere di **ditte differenti**.

Rete. Per ogni proprietà p_j si creano t **nodi-gadget** $d_{j,k}$ (uno per ogni ditta):

- $S \rightarrow s_i$ con capacità = numero di proprietà che s_i sa misurare;
- $s_i \rightarrow d_{j,k}$ (capacità 1) se s_i è prodotto dalla ditta k e sa misurare p_j ;
- $d_{j,k} \rightarrow p_j$ con capacità 1 $\leftarrow$ **la chiave**: passa al più un'unità di flusso per coppia (proprietà, ditta), quindi due strumenti della stessa ditta non possono misurare la stessa proprietà;
- $p_j \rightarrow P$ con capacità r .

Risposta. Fattibile $\iff$ il flusso raggiunge il valore target ($n \cdot r$). Con $|V| = n + m + t + 2$ e $|E| = O(n + m + nm + tm)$, il costo è $O(m^2(n + t)r)$.

D.3 Matching su un solo insieme

Coppie "inquadrabili" [18-06-04 · Es.2]

Traccia. Due interi sono *inquadrabili* se la loro somma è un quadrato perfetto (es. $4 + 5 = 3^2$). Dato X di n interi distinti, trovare il **massimo numero di coppie inquadrabili disgiunte** (ogni valore usato al più una volta).

Soluzione. È un matching massimo, ma su un **unico insieme** di elementi: si "sdoppia" ogni elemento in a_i e b_i .

- $s \rightarrow a_i$ (capacità 1), $b_j \rightarrow t$ (capacità 1);
- $a_i \rightarrow b_j$ (capacità 1) $\iff X[i] < X[j]$ (la disuguaglianza stretta evita di contare due volte la stessa coppia) e $\lfloor \sqrt{X[i] + X[j]} \rfloor = \sqrt{X[i] + X[j]}$.

Risposta. Il flusso massimo è il numero di coppie; le coppie scelte sono gli archi $a_i \rightarrow b_j$ con flusso 1. Con $|V| = 2n + 2$, $|E| = O(n^2)$ e $|f^*| \leq n/2$, il costo è $O(n^3)$ con Ford-Fulkerson (Hopcroft-Karp farebbe $O(n^2\sqrt{n})$).

Ottimizzazione: si possono eliminare i nodi a_i senza archi uscenti e i b_i senza archi entranti.

⚠ **Due soluzioni sbagliate molto comuni su questo esercizio:**

- **Greedy** (accoppia la prima coppia valida che trovi): su $X = [1, 15, 3, 10]$ "consuma" $1 + 15$ e restituisce 1, mentre l'ottimo è 2 ($1 + 3$ e $15 + 10$).
- **DP sugli intervalli** $DP[i][j]$: su $X = [1, 3, 6, 19]$ restituisce 1 invece di 2 ($1 + 3$ e $6 + 19$). **I sottoproblemi non sono indipendenti**, quindi non vale la sottostruttura ottima.

D.4 Cammini arco-disgiunti

I figli della signora Eris [16-01-26 · Es.2]

Traccia. Due fratelli devono andare da casa a scuola **senza percorrere nessun tratto di strada in comune**. È possibile?

Soluzione. Rete di flusso con **capacità 1 su ogni arco**, sorgente = casa, pozzo = scuola. Per il teorema di Menger, il **flusso massimo è pari al numero di cammini arco-disgiunti**. Risposta: **true** $\iff |f^*| \geq 2$.

Ottimizzazione: basta fermare Ford-Fulkerson dopo **2 iterazioni** (non serve il flusso massimo, serve solo sapere se è ≥ 2) $\rightarrow O(m + n)$.

D.5 Teoria del flusso e ricalcolo incrementale

Archi upward-critical e downward-critical [17-01-10 · Es.3]

Traccia. Data una rete $G = (V, E, s, t, c)$ con capacità intere, un arco (u, v) è *upward-critical* se **aumentando** $c(u, v)$ di 1 il flusso massimo **aumenta**; è *downward-critical* se **diminuendo** $c(u, v)$ di 1 il flusso massimo **diminuisce**. Esibire (se esistono) reti con/senza archi di ciascun tipo.

Soluzione.

1. *Esiste un upward-critical:* $A \xrightarrow{1} B \xrightarrow{2} C$ — l'arco (A, B) è upward-critical (è il collo di bottiglia).
2. *Nessun upward-critical:* $A \xrightarrow{1} B \xrightarrow{1} C$ — aumentare la capacità di un solo arco non aumenta il flusso, perché l'altro resta il collo di bottiglia.
3. *Tutti downward-critical:* la stessa rete $A \xrightarrow{1} B \xrightarrow{1} C$.
4. *Nessun downward-critical:* solo con reti degeneri in cui il flusso massimo è 0 (es. $A \rightarrow B$ e $C \rightarrow D$ scollegati).

Il risultato interessante: se $|f^*| > 0$, un arco downward-critical **esiste sempre**. *Dimostrazione per assurdo.* Se nessun arco fosse downward-critical, nessun arco sarebbe saturo; allora tutti gli archi originali comparirebbero nella rete residua. Ma poiché $|f^*| > 0$ esiste un cammino $s \rightsquigarrow p$ nel grafo originale, che sarebbe quindi un **cammino aumentante** nella rete residua — assurdo, perché il flusso era massimo. ■

Ricalcolo del flusso dopo la riduzione di una capacità [15-12-17 · Es.4]

Traccia. Dato un flusso massimo f già calcolato e un arco (u, v) con $c[u, v] > 0$, ridurre $c[u, v]$ di 1 e **ricalcolare il flusso massimo** senza ripartire da zero.

Soluzione.

1. Se l'arco **non è saturo** ($f[u, v] < c[u, v]$), non c'è nulla da fare: il flusso resta valido e massimo.
2. Altrimenti si riduce $f[u, v]$ di 1. Questo però **viola la conservazione del flusso** in u e in v : con una **reduceDFS** si riduce di 1 il flusso lungo **un cammino** $s \rightsquigarrow u$ e lungo **un cammino** $v \rightsquigarrow p$.
3. Ora il flusso è valido, ma vale $|f| - 1$. Basta cercare **un solo cammino aumentante** sulla rete residua: se esiste, il flusso torna al valore originale; altrimenti il nuovo massimo è $|f| - 1$.

Complessità. $O(m + n)$ per le operazioni sui cammini ($O(n^2)$ con matrice di adiacenza) — non serve rieseguire Ford-Fulkerson.

E. Greedy

Da fare sempre: o si **dimostra** l'ottimalità (quasi sempre con l'**argomento di scambio**: si prende una soluzione ottima che differisce dalla greedy, si scambiano due elementi adiacenti/diversi, si mostra che la soluzione non peggiora → assurdo), oppure si esibisce un **controesempio** piccolo e verificabile a mano.

E.1 Ordinamento per rapporto o per valore

File su nastro sequenziale [14-06-16 · Es.3] [17-01-10 · Es.4]

Traccia. n file: il file i ha lunghezza ℓ_i e probabilità di accesso p_i . I file vanno scritti su un nastro **sequenziale** secondo una permutazione π . Il costo di lettura dell' i -esimo file della permutazione è la somma delle lunghezze dei file che lo precedono (incluso):

$$c_i = \sum_{j=1}^i \ell_{\pi(j)} \quad C = \sum_{i=1}^n p_{\pi(i)} \cdot c_i$$

Minimizzare il costo atteso C . Si valutino gli algoritmi: (a) ordinare per lunghezza ℓ_i crescente; (b) ordinare per probabilità p_i decrescente.

Soluzione. Entrambi gli algoritmi proposti sono sbagliati:

- **per lunghezza crescente** — controesempio: file $(\ell = 1, p = 0.1)$ e $(\ell = 2, p = 0.9)$. L'ordine $(1, 2)$ costa $1(0.1) + 3(0.9) = 2.8$; l'ordine $(2, 1)$ costa $2(0.9) + 3(0.1) = 2.1$. ×
- **per probabilità decrescente** — controesempio: file $(\ell = 10, p = 0.6)$ e $(\ell = 1, p = 0.4)$. L'ordine $(1, 2)$ costa $10(0.6) + 11(0.4) = 10.4$; l'ordine $(2, 1)$ costa $1(0.4) + 11(0.6) = 7.0$. ×

Algoritmo corretto: ordinare per rapporto ℓ_i/p_i CRESCENTE. Costo $O(n \log n)$.

Dimostrazione (scambio di adiacenti). Sia π ottima e siano $a = \pi(i)$, $b = \pi(i+1)$ consecutivi con $\frac{\ell_a}{p_a} > \frac{\ell_b}{p_b}$. Scambiandoli, il costo di a **aumenta** di ℓ_b e quello di b **diminuisce** di ℓ_a ; tutti gli altri costi restano invariati. La variazione totale è $p_a \ell_b - p_b \ell_a$, che deve essere ≥ 0 perché π è ottima; da cui

$$\frac{\ell_a}{p_a} \leq \frac{\ell_b}{p_b}$$

che contraddice l'ipotesi. ■

(Nota: questo identico esercizio è stato dato due volte, a due anni di distanza. È il candidato numero uno per una ricomparsa.)

Negazione k -massimale [18-06-04 · Es.1]

Traccia. Dato X di n interi **distinti** e un intero k , scegliere esattamente k indici da **negare** (cambiare di segno) massimizzando la somma degli elementi del vettore risultante $\overline{X}_S$, dove

$$\overline{X}_S[i] = \begin{cases} -X[i] & i \in S \\ X[i] & i \notin S \end{cases}$$

Soluzione greedy. Ordinare e **negare i k valori più bassi**. Costo $O(n \log n)$ ($O(n)$ se già ordinato).

Dimostrazione (per assurdo/scambio). Sia S l'insieme dei k valori più bassi e M un ottimo con $M \neq S$. Esistono $s \in S \setminus M$ e $m \in M \setminus S$, con $X[m] > X[s]$ (perché s è fra i k più bassi e i valori sono distinti). Posto $M' = M - \{m\} \cup \{s\}$:

$$\sum_{i=1}^n \overline{X}_{M'}[i] = \sum_{i=1}^n \overline{X}_M[i] + 2X[m] - 2X[s] > \sum_{i=1}^n \overline{X}_M[i]$$

(il fattore 2 perché m passa da $-X[m]$ a $+X[m]$ e s viceversa) — assurdo, perché M era ottimo. ■

E.2 Intervalli: copertura minima

Copertura di un insieme di punti con intervalli unitari [14-12-19 · Es.1]

Traccia. Dato un insieme V di n numeri reali, trovare il **minimo numero di intervalli unitari** $[x, x+1]$ che li ricoprono tutti.

Soluzione. Ordinare V . Il primo intervallo è $[V[1], V[1] + 1]$ (**far partire l'intervallo dal punto più a sinistra non ancora coperto è sempre ottimo**). Poi si scandisce: appena si trova un $V[i]$ che cade fuori dall'ultimo intervallo, se ne apre uno nuovo che parte da $V[i]$. Costo $O(n \log n)$, dominato dall'ordinamento.

Dimostrazione: argomento di scambio sulla scelta greedy — spostare a destra l'inizio del primo intervallo non può coprire più punti.

High Line: irrigatori (copertura di un segmento) [17-06-29 · Es.3]

Traccia. Un parco lineare lungo L ; l'irrigatore i è alla distanza $D[i]$ e ha raggio $R[i]$, quindi copre $[D[i] - R[i], D[i] + R[i]]$. Minimo numero di irrigatori per bagnare **tutto** il segmento $[0, L]$, oppure -1 se impossibile.

Soluzione. ⚠ **Variante diversa dalla precedente:** qui va coperto un **segmento continuo**, non dei punti. Si ordinano gli intervalli per estremo **sinistro** crescente. Ad ogni passo, fra tutti gli intervalli che **iniziano prima o esattamente dove finisce la copertura attuale (last)**, si sceglie quello con l'**estremo destro massimo**. Se nessun intervallo interseca $last \rightarrow -1$ (buco non copribile). Alla fine si verifica $last \geq L$. Costo $O(n \log n)$.

Dimostrazione: sostituire l'intervallo scelto con quello di estremo destro più a destra dà una soluzione della stessa cardinalità che copre almeno quanto la precedente.

Minimo sottoinsieme di intervalli con la stessa copertura [16-01-07 · Es.3]

Traccia. Dato un insieme di intervalli, trovarne il **minimo sottoinsieme che copre esattamente la stessa porzione di retta**.

Soluzione. Ordinare per inizio crescente (e a parità, per fine decrescente); scartare gli intervalli **contenuti** in un altro. La versione corretta richiede un **"backtrack iterativo"**: prima di inserire l'intervallo i , si rimuovono dalla soluzione gli ultimi intervalli inseriti che sono diventati inutili (quelli con $a[i] \leq b[S[j-1]]$, cioè completamente scavalcati). Costo $O(n \log n)$.

E.3 Greedy che NON funziona

Sotto-categoria da conoscere bene: in questi esami è **molto frequente** che venga chiesto di *smontare* un greedy plausibile. Il controesempio va tenuto piccolissimo e verificato numericamente.

Colorazione greedy dei grafi [15-12-17 · Es.1]

Traccia. L'algoritmo greedy che scorre i vertici e assegna a ciascuno il **primo colore libero** usa al più $d + 1$ colori (d = grado massimo). Ma è ottimo?

Soluzione. No. Il numero di colori dipende **dall'ordine di visita**. Controesempio: si costruisce un grafo **bipartito** (quindi 2-colorabile) e un ordine $v_1, \dots, v_n$ "a zig-zag" che forza il greedy a usare $n/2$ colori. Il greedy garantisce solo il limite superiore $d + 1$, non l'ottimalità.

Matching di peso massimo per peso decrescente [15-07-14 · Es.3]

Traccia. Un insieme di archi $A \subseteq E$ è *univoco* se nessun nodo ha due archi incidenti in A (cioè è un matching). Trovare l'insieme univoco di **peso massimo**. Funziona il greedy che prende gli archi in ordine di peso decrescente?

Soluzione. No. Controesempio (un cammino):

$$u \xrightarrow{2} v \xrightarrow{3} w \xrightarrow{2} x$$

Il greedy prende per primo l'arco di peso 3, cioè (v, w) , e a quel punto non può più prendere nulla $\rightarrow$ totale 3. L'ottimo è $\{(u, v), (w, x)\} \rightarrow$ totale 4.

Soluzione corretta proposta: **backtracking** su prendi/non-prendi ciascun arco, $O(2^m)$.

Greedy sui quadrati [16-01-26 · Es.4]

Il greedy "sottrai sempre il quadrato più grande possibile" fallisce su 18: darebbe $16 + 1 + 1$ (3 termini) mentre l'ottimo è $9 + 9$ (2 termini). $\rightarrow$ Va usata la DP, vedi B.4.

Greedy sulle coppie inquadrabili [18-06-04 · Es.2]

Il greedy che accoppia la prima coppia valida fallisce su $X = [1, 15, 3, 10]$. $\rightarrow$ Va usato il flusso, vedi D.3.

Greedy su scale e serpenti [17-06-29 · Es.2]

"Segui le scale, evita i serpenti" fallisce: possono esserci 6 caselle consecutive tutte serpenti, o cicli scala-serpente. $\rightarrow$ Va usata la BFS, vedi B.7.

F. Backtracking ed enumerazione

Schema base. Una funzione ricorsiva `rec(S, i, ...)` con: (1) **condizione di terminazione** $\rightarrow$ stampa la soluzione; (2) ciclo sulle **scelte ammissibili** per la posizione i ; (3) registrazione della scelta in $S[i]$ e chiamata ricorsiva su $i + 1$. La complessità è quasi sempre **superpolinomiale** e va dichiarata esplicitamente.

⚠ Errore classico (segnalato dal docente): separare il test "ho raggiunto l'obiettivo" dal test "ho esaurito gli elementi" fa stampare **la stessa soluzione più volte**. I due test vanno annidati.

Stampare tutte le disposizioni del domino $2 \times n$ [15-12-17 · Es.3b]

Traccia. Stampare tutti i modi di ricoprire una scacchiera $2 \times n$ con tessere di domino.

Soluzione. Backtracking sulla matrice $2 \times n$ delle scelte: in ogni posizione i si può mettere **una tessera verticale** (ricorsione su $i - 1$) oppure **due tessere orizzontali** in i e $i - 1$ (ricorsione su $i - 2$). Complessità $O(n \cdot 2^n)$.

(Il conteggio, invece di essere fatto con l'enumerazione, si fa con Fibonacci in $O(n)$: vedi [B.6](#).)

Stampare tutte le decomposizioni di n in somma di quadrati [\[16-01-26 · Es.4b\]](#)

Traccia. Stampare **tutte** le decomposizioni di n in somma di quadrati.

Soluzione. Backtracking con un parametro aggiuntivo $max = \text{valore massimo ancora utilizzabile}$. Serve a imporre che i termini vengano scelti in ordine non crescente, evitando di stampare la stessa decomposizione in tutte le sue **permutazioni**. Complessità superpolinomiale.

Insieme univoco di archi di peso massimo [\[15-07-14 · Es.3b\]](#)

Traccia. Trovare il matching di peso massimo (dopo aver smontato il greedy).

Soluzione. Backtracking su ogni arco: prendi / non prendi, verificando l'ammissibilità (nessun nodo con due archi incidenti). Complessità $O(2^m)$.

Stampare tutte le stringhe bilanciate di parentesi [\[17-09-04 · Es.4\]](#)

Traccia. Dato n , stampare tutte le stringhe di $2n$ caratteri con n coppie di parentesi bilanciate. Per $n = 2$: $(())$ e $()()$.

Soluzione. Si mantengono due contatori: $open$ = parentesi ancora da aprire, $close$ = parentesi aperte ancora da chiudere.

```
printparrec(L, i, open, close)
  if open + close = 0 then
    print L
  else
    if open > 0 then
      L[i] ← '('
      printparrec(L, i+1, open-1, close+1)
    if close > 0 then
      L[i] ← ')'
      printparrec(L, i+1, open, close-1)
```

Chiamata iniziale: `printparrec(L, 0, n, 0)`. Complessità esponenziale: il numero di soluzioni è il **numero di Catalan** $C_n = \frac{1}{n+1} \binom{2n}{n}$.

Stampare tutti i sottoinsiemi con somma w [\[18-06-04 · Es.3\]](#)

Traccia. Dato X di n interi distinti e un target w , stampare **tutti** i sottoinsiemi di X la cui somma è esattamente w .

Soluzione. Enumerazione di tutti i sottoinsiemi con una maschera booleana $S[1..n]$:

```
printSums(X, n, v, i, S)
  if i = n+1 then
    if v = 0 then                                     ← i due test ANNIDATI (vedi l'errore classico qui sopra)
      printVector(X, S, n)
    return
  foreach c ∈ {0,1} do
    S[i] ← c
    printSums(X, n, v - c·X[i], i+1, S)
```

Albero di decisione di dimensione 2^n , e ogni stampa costa fino a $n \rightarrow$ complessità $O(n \cdot 2^n)$.

Bandiere con colori abbinabili [\[19-06-06 · Es.B2\]](#)

Traccia. Bandiere di n strisce, k colori; una matrice booleana simmetrica $A[c_1][c_2]$ dice quali colori sono **abbinabili** (con $A[c][c] = \text{false}$, quindi mai due strisce contigue uguali). Stampare **tutte** le bandiere valide.

Soluzione. Backtracking; l'insieme delle **scelte ammissibili** per la striscia i è:

- **tutti** i k colori, se $i = 1$;
- tutti i colori j tali che $A[j][S[i-1]] = \text{true}$, altrimenti.

```
printFlagsRec(A, S, i, n, k)
  if i = n+1 then
    print S
  else
    for j ← 1 to k do
      if i = 1 or A[j][S[i-1]] then
        S[i] ← j
        printFlagsRec(A, S, i+1, n, k)
```

Complessità. $O(n \cdot k^n)$, superpolinomiale: il fattore n è la stampa, mentre k^n è il caso pessimo in cui la matrice ha **false** solo sulla diagonale.

G. Array, vettori, ordinamento, divide et impera

Esercizi che **non richiedono una tecnica avanzata**: si risolvono con una scansione lineare, un ordinamento, un dizionario/hash o una ricerca binaria. Sono in genere gli Esercizio 1 o 2 (i più leggeri). Il punto è **riconoscere** che non serve nient'altro — e giustificare bene correttezza e complessità.

Più lungo sottovettore $0^k 1^k$ [17-01-10 · Es.1]

Traccia. Vettore binario V . Trovare il più lungo sottovettore **contiguo** formato da k zeri seguiti da k uni. (Es.: $00111111100011110000 \rightarrow 6$.)

Soluzione. Semplice **scansione lineare** con due contatori: $count_0$ (zeri consecutivi correnti) e $count_1$ (uni consecutivi successivi). Quando dopo degli uni si torna a leggere uno 0, entrambi i contatori si azzerano e riparte il conteggio. Il risultato è

$$2 \cdot \max_i \left(\min(count_0, count_1) \right)$$

Complessità. $\Theta(n)$.

Verificare se un vettore è un min-heap [19-06-06 · Es.B1]

Traccia. Dato V di n interi distinti, dire se rappresenta un **min-heap** memorizzato in un vettore.

Soluzione. Per ogni i da 2 a n , verificare $V[\lfloor i/2 \rfloor] \leq V[i]$ (il padre non deve mai essere maggiore del figlio). Basta un solo controesempio per restituire **false**. **Complessità.** $\Theta(n)$.

Ordinare un vettore con solo m valori distinti ($m \ll n$) [15-09-01 · Es.1]

Traccia. Ordinare un vettore di n elementi che contiene solo m **valori distinti**, in meno di $O(n \log n)$.

Soluzione A. Dizionario implementato con **albero RB**: si inserisce ogni elemento contando le occorrenze — $O(n \log m)$, perché l'albero ha solo m chiavi — poi si itera in ordine riscrivendo il vettore. Totale $O(n \log m)$.

Soluzione B. Tabella hash per contare le occorrenze in $O(n)$, poi si ordinano le **sole m chiavi** in $O(m \log m)$. Totale $O(n + m \log m)$ — migliore.

Due valori distinti con la stessa frequenza? [17-09-04 · Es.2]

Traccia. Dato A di n interi (**non** necessariamente distinti), esistono due valori distinti che compaiono lo **stesso numero di volte**? (Es.: $[3, 1, 3, 4, 1, 4, 5] \rightarrow \text{true}$; $[3, 1, 3, 3, 1] \rightarrow \text{false}$.)

Soluzione $O(n \log n)$. Ordina A ; conta le occorrenze di ogni valore in un vettore B ; ordina B — con **CountingSort** in $O(n)$, dato che le frequenze stanno fra 1 e n — e cerca due elementi **consecutivi uguali** in B . Il costo è dominato dal primo ordinamento.

Soluzione $O(n)$. **Tabella hash** per contare le frequenze (elimina il primo ordinamento), poi una seconda struttura per rilevare frequenze duplicate.

Massimo k tale che B^k è sottosequenza di A [15-06-08 · Es.4]

Traccia. B^k è la stringa B con **ogni carattere ripetuto k volte**. Trovare il massimo k tale che B^k è sottosequenza di A .

Soluzione. Una funzione **isSubsequence greedy a due indici** verifica in $O(n)$ se B^k è sottosequenza di A : si avanza su A e si consuma $B[\lceil j/k \rceil]$. Si itera su k crescente finché il test ha successo → **complessità** $O(n^2/m)$.

Riordinare una matrice ordinata per righe e colonne [16-07-06 · Es.2]

Traccia. Una matrice $m \times n$ è ordinata per righe e colonne ($A[i, j] \leq A[i, j+1]$ e $A[i, j] \leq A[i+1, j]$). Riordinarla globalmente (riga per riga).

Soluzione. Copiare tutti gli $n \cdot m$ elementi in un vettore, ordinarlo, riscriverlo riga per riga. **Complessità.** $O(nm \log(nm))$.

(La proprietà di ordinamento parziale non aiuta a fare meglio in modo semplice: ogni elemento è già $\geq$ di quelli che lo precedono in riga e in colonna, ma questo non induce un ordine totale.)

Trovare un picco in $O(\log n)$ — divide et impera [15-09-01 · Es.3]

Traccia. Un elemento $A[i]$ è un **picco** se è $\geq$ di entrambi i suoi vicini. (a) Dimostrare che un picco esiste **sempre**. (b) Trovarne uno in $O(\log n)$.

Soluzione.

(a) Il **massimo** del vettore è per definizione $\geq$ di entrambi i vicini, quindi è un picco. Un picco esiste sempre.

(b) **Ricerca binaria:** sia $m = \lfloor (i+j)/2 \rfloor$. Se $A[m]$ è un picco → restituiscilo. Altrimenti, se $A[m-1] > A[m]$ cerca **a sinistra** (in quella metà c'è sicuramente un picco, perché la sequenza sta salendo verso sinistra), altrimenti cerca **a destra**. **Complessità.** $O(\log n)$.

Appendice: indice per esame

Per ritrovare rapidamente un esercizio partendo dalla data del compito.

Esame	Es. 1	Es. 2	Es. 3	Es. 4
14-06-16	Ricorrenza $2T(n/8) + \sqrt[3]{n}$ [A.1]	Componenti connesse acicliche [C.1]	Greedy file su nastro [E.1]	DP auto elettrica [B.5]
14-07-21	Ricorrenza con min , ipotesi rafforzata [A.3]	Vertice principale [C.2]	Flusso sensori backup [D.1]	DP sottosequenza palindroma [B.2]
14-12-19	Greedy intervalli unitari [E.2]	Flusso piano di studi [D.1]	DP sottoseq. crescente distinta [B.1]	DP allineamento con dash [B.1]
15-02-02	Ricorrenza 4 chiamate + n^2 [A.2]	Contare vertici principali (SCC) [C.2]	Flusso parole con cubi [D.1]	DP salvadanaio [B.4]
15-06-08	Ricorrenza 3 chiamate + n^3 [A.2]	Ordine di eliminazione (DFS) [C.1]	DP resto con $\leq k$ monete [B.4]	Sottosequenza B^k [G]
15-07-14	Ricorrenza, limite inferiore [A.1]	Cammini minimi Alice/Bob/Carl [C.3]	Greedy matching: controes. + backtrack [E.3][F]	DP calcetto (2 squadre) [B.4]

Esame	Es. 1	Es. 2	Es. 3	Es. 4
15-09-01	Ordinare con m valori distinti [G]	Flusso conferenza SIESTA [D.1]	Divide et impera: picco [G]	DP sottostringa palindroma [B.2]
15-12-17	Controesempio colorazione greedy [E.3]	DP $\leq k$ elementi consecutivi [B.5]	Domino $2 \times n$: conta [B.6] + stampa [F]	Flusso: ricalcolo incrementale [D.5]
16-01-07	Merge/Quick/Insertion su input 5-crescente [A.4]	Cammino minimo in albero allopatato [C.3]	Greedy copertura minima intervalli [E.2]	DP taglio ottimo stringa [B.3]
16-01-26	Due ricorrenze (una con ipotesi rafforzata) [A.3]	Flusso: cammini arco-disgiunti [D.4]	DP shuffle di stringhe [B.1]	DP somma di quadrati [B.4] + stampa [F]
16-07-06	Ricorrenza $2T(n/\sqrt{2} - 5) + n^{\pi/2}$ [A.1]	Ordinare matrice per righe/colonne [G]	DP Route 66 (distanza minima) [B.5]	DAG: caccia al tesoro, valore atteso [C.4]
16-08-29	Ricorrenza 4 chiamate + $\sqrt{n}$ [A.2]	DAG: cammino con etichetta data [C.4]	Flusso: Consiglio CdL (quote) [D.2]	DP: contare sottosequenze [B.1]
17-01-10	Sottovettore $0^k 1^k$ [G]	DP supersequenza comune minimale [B.1]	Flusso: archi up/down-critical [D.5]	Greedy file su nastro (bis) [E.1]
17-01-24	Ricorrenza da codice (crazy) [A.4]	Bipartizione wrestler [C.1]	Flusso: segherie [D.1]	DP: decomposizione in palindromi [B.2][B.3]
17-06-29	Ricorrenza 3 chiamate + n^2 [A.2]	Scale e serpenti (BFS) [B.7]	Greedy: irrigatori High Line [E.2]	Flusso: CERN con gadget ditte [D.2]
17-09-04	Ricorrenza 3 chiamate + n^2 [A.2]	Frequenze uguali (hash) [G]	DAG: snowboard, cammino discendente [C.4]	Backtracking: parentesi bilanciate [F]
18-06-04	Greedy: negazione k -massimale [E.1]	Flusso: coppie inquadabili [D.3]	Backtracking: sottoinsiemi con somma w [F]	DP: somma (k, w) -vincolata [B.4]
19-06-06	Verifica min-heap [G]	Backtracking: bandiere colorate [F]	DP: cammini su griglia con budget [B.6]	(assente)

Frequenza per macro-categoria

Macro-categoria	Esercizi	%
B — Programmazione dinamica e memoization	21	30%
A — Ricorrenze e analisi di complessità	14	20%
C — Grafi	13	18%
D — Flusso massimo e matching	11	15%
E — Greedy	7	10%
F — Backtracking ed enumerazione	6	8%
G — Array, ordinamento, divide et impera	6	8%

(La somma supera 71 perché alcuni esercizi appartengono a due categorie: tipicamente "risolvi con la DP" + "stampa tutte le soluzioni con il backtracking", oppure "smona il greedy" + "proponi l'algoritmo corretto".)

Come si presenta un esame tipico

Guardando la tabella per esame emerge una struttura molto regolare:

- **Esercizio 1** (≥ 6 punti): quasi sempre una **ricorrenza** da risolvere per sostituzione, oppure un esercizio facile su vettori.
- **Esercizio 2** ($\geq 6-7$ punti): un **grafo** (visita, raggiungibilità) oppure una DP semplice su stringhe.

- **Esercizio 3** (≥ 8 punti): **flusso massimo** oppure **greedy con dimostrazione/controesempio**.
- **Esercizio 4** (≥ 10 punti): **programmazione dinamica**, spesso con una seconda parte di **backtracking** per stampare tutte le soluzioni.

Le tre cose che ricorrono più spesso in assoluto e che vale la pena avere in memoria muscolare: (1) lo **schema di dimostrazione per sostituzione** con l'ipotesi rafforzata $T(n) \leq cn - b$; (2) il **template di costruzione di una rete di flusso** (nodi, capacità, valore target, complessità $O(|f^*|(|V| + |E|))$ di Ford-Fulkerson); (3) l'**argomento di scambio** per dimostrare un greedy — e il riflesso di cercare un controesempio piccolo quando l'esercizio *ti chiede* se un greedy funziona (spesso la risposta è no).